

Airport Modelling Project

Group 41: Requirements Analysis

1 Introduction

As global air travel demand increases, airport infrastructure must maximise efficiency while maintaining safety. This project aims to develop a software system for airport operations staff to model aircraft arrivals and departures under dynamic conditions. By simulating various runway configurations, the system enables the analysis of throughput and delays to inform operational strategy to maximise revenue and customer satisfaction. The system operates as a stand-alone, self-contained environment, relying solely on user inputs and internal state. This ensures all simulation results are reproducible and comparable across different runs, enhancing the reliability of conclusions drawn. This document formalises the client's needs into verifiable requirements, ensuring full traceability between user goals and the final delivered system.

2 User Requirements Design

The following requirements are formulated using the EARS format to reduce ambiguity, vagueness and wordiness. [1] They provide a clear and verifiable foundation for the system's functional scope while deferring technical implementation decisions to Section 4. To enhance readability and space efficiency within the tables, the standard "The system shall..." prefix is omitted but remains implied for almost every requirement, except those starting with the structural keyword "If".

Table 1: User requirements. Note: All IDs in this table correspond to the UR-FR-x prefix (e.g. 1 = UR-FR-1). Priority (abbreviated to Pr.) is defined by the MoSCoW method: M (Must), S (Should), C (Could), W (Won't).

ID	Pr.	User Requirement	Justification and Source
Configuration Mode			
1	M	Allow configuration of 1-10 available runways.	A "primary input parameter", limited "between 1 and 10".
2	M	Allow configuration of inbound and outbound aircraft flow rates from 0 to 100/hr each.	Identified as a "primary input parameter". Combined 200/hr max flow rate allows global peak of 185 ops/hr. [2]
3	M	Allow configuration of each runway's initial mode as Landing, Take-Off, or Mixed Mode.	User must be able to "configure the... Runway Operating Mode" between these states. Model requires initial modes.
4	M	Allow configuration of each runway's initial status as Available, Runway Inspection, Snow Clearance or Equipment Failure.	User must be able to "configure the... Runway Operating Status" between these statuses. Model requires initial statuses.
5	M	Allow configuration of max wait time for departing aircraft before departures are cancelled.	Spec requires a "configurable maximum wait time" to "model the impact of runway closures".
6	M	Allow scheduling of runway status changes and aircraft emergencies at specific simulation timestamps.	Required to achieve client's objective to "monitor operational changes" during "scenarios such as runway closures". Enables comparing responses to determine best strategy.
7	M	If the user enters invalid configuration values, then the system shall reject the input and present a corrective error message.	Ensures data integrity; prevents crashes from invalid parameters. Satisfies "useful validation and error handling".
8	S	Allow the user to save current and retrieve and modify previously used airport configurations.	Primary objective is to "model various airport scenarios". Allows user to "compare multiple variations" more quickly.
9	C	Allow user to enable automatic generation and configure rates of events such as runway failure.	Addresses optional "statistical event modelling". Automation ensures data is representative over long periods by matching the client's real-world event rates.
Simulation Mode			
10	M	Execute simulation logic using the user-defined configuration parameters and scheduled events.	Ensures the simulation state is derived directly from user inputs, allowing user to "model various airport scenarios".
11	M	Generate aircraft uniformly based on the configured flow rate, applying a Normal Distribution offset ($\sigma = 5$ mins) to determine actual arrival/departure readiness times.	Uniform generation is interpreted from "flow" rate to ensure reproducible comparisons, actual times are "normally distributed" for a more representative, stochastic model.
12	M	Use discrete time-steps (ticks) to track fuel consumption, wait-time durations and cancellation thresholds.	Spec: "aircraft consume fuel at a constant rate", time-stepping is required to decrement fuel, and increment wait counters to determine if departures should be cancelled.
Simulation Mode: Arrivals Logic			
13	M	Ensure inbound aircraft land immediately if the holding queue is empty and a suitable runway is available; otherwise, they enter the holding pattern.	Spec: "land immediately if there is no other traffic to consider...otherwise...enter...holding pattern". Ensures aircraft don't enter holding pattern unnecessarily.
14	M	Prioritise landings by emergency status first and then by the order in which they entered the holding pattern.	Spec: "1. Emergency... 2. In queue order". Ensures safety-critical events are handled first, making the model more representative and useful.
Continued on next page...			

Table 1

ID	Pr.	User Requirement	Justification and Source
15	M	Ensure aircraft in the holding pattern maintain a minimum vertical separation of 1000ft from other holding aircraft.	Specification states: "Aircraft in holding patterns must be separated vertically by at least 1000ft". Required to ensure the model is safe and representative.
16	M	Ensure arriving aircraft are cleared to land, or otherwise diverted, before remaining fuel reaches 10 minutes.	Spec: "aircraft must land, or be diverted elsewhere, before the fuel reaches 10 minutes' worth". Ensures fuel exhaustion is avoided to make model more realistic.
17	M	Ensure each arriving aircraft is assigned an initial, limited fuel level uniformly distributed between 20 and 60 minutes of flight time.	Spec: "uniformly distributed between 20-60 minutes' worth". Ensures a realistic variety of endurance, allowing for analysis of fuel-related emergencies and diversions.
Simulation Mode: Departures Logic			
18	M	Ensure aircraft join the back of the take-off queue in order they receive departure clearance.	Spec: "Aircraft join the back of the take-off queue once... clearance to take-off". Ensures fair take-off ordering.
19	M	Ensure aircraft at the front of the queue take-off when a runway is available and clear of traffic.	Spec: "When a runway slot becomes available... front of the queue will proceed to take off". Ensures model is safe.
Simulation Mode: Runway Mode Logic			
20	M	Enforce that only a single aircraft may occupy an active runway, including the immediate approach and departure zones.	Explicitly stated: "only a single aircraft may occupy a runway". Essential for "maintaining safety", ensuring the results are representative of real-world operations.
21	M	Divide mixed-mode availability between landings and take-offs, prioritising emergency arrivals.	Spec: "availability must be divided", ensures traffic doesn't stall. Priority logic ensures emergencies are modelled safely so the output is representative.
22	M	Enforce a minimum runway occupancy time for every take-off and landing operation.	Required to provide representative data. Without this, the model cannot accurately calculate effects on "throughput".
23	M	Re-sequence queues and landing priorities in response to runway status changes.	Spec states: "Model the impact of runway closures on the holding queue and take-off queue".
Simulation Mode: User Interface			
24	M	Provide a control to start the simulation and a progress indicator.	Ensures the user knows the simulation is active if real-time views are disabled.
25	S	Provide a real-time model via data tables (listing aircraft, queues, runway zones and running averages of tracked metrics) and simulated time.	Necessary to allow the operator to "monitor aircraft throughput" while the simulation is running and to see effects of operational changes in real time.
26	S	Provide visual alerts, e.g. red colour coding, when an aircraft enters an emergency status.	A key priority for the operator is to "monitor scenarios such as... fuel emergencies".
27	S	Provide controls to pause, resume and adjust simulated time scale, and to end the simulation early before the configured simulation duration.	Necessary for usability and to allow operators to "respond effectively to situations", e.g. by pausing and considering what action to take.
28	S	Allow the user to manually change emergency status of aircraft, and the status and modes of runways mid-simulation.	Allows the user to dynamically model the "impact of runway closures" and aircraft emergencies.
29	C	Provide a 2D real-time graphical representation of the simulation, showing numbers in queues and aircraft moving between airport zones.	"Visual representation" is defined as an optional feature in the specification, as it is beneficial for usability, but not essential for the core logic of the simulation.
Output Modes			
30	M	Compute the total throughput (take-offs and landings/hr) of the airport simulation.	"Aircraft throughput is a key priority for airport managers", therefore it is a key metric to help compare configurations.
31	M	Calculate the maximum length and average wait times for both the take-off and holding queues.	Required in the specification, priorities 1b and 2b in the "output" section.
32	M	Compute the maximum and average delay by comparing scheduled arrival/departure times against actual times.	Required in the specification, priorities 1c and 2c in the "output" section.
33	M	Report the total number of cancelled outbound flights (due to wait limits) and diverted inbound flights (due to fuel/capacity).	Required in the specification, priorities 4b and 5a in the "output" section. Required to link to loss of revenue and customer satisfaction.
34	S	Provide the option to save results and enter Comparison Mode after a simulation, and to enter Comparison Mode from Home Mode.	Enables the user to "compare multiple variations", which is essential for evaluating the relative success of different strategies.
35	S	If additional modelling metrics are enabled, then the system shall provide extra information, e.g. range and variance.	Addresses the optional "additional modelling metrics" feature. Provides data to help users "explain the distribution" and understand variability and risk in different scenarios.

2.1 Validation Process

To ensure high-quality User Requirements, the team followed a systematic review process led by the Project Manager. Each requirement was validated against three core criteria: origin, appropriateness and verifiability. Firstly, user requirements were derived directly from the client specification or informed by additional research, such as IATA traffic data, to ensure they represent client needs. Secondly, the team prioritised the requirements using the MoSCoW method, to ensure "Must-have" core logic is implemented first to satisfy the client's primary objectives. By deferring optional features until the core system is complete, we ensure the final product remains appropriate and functional even if technical challenges or delays occur during the project lifecycle. Finally, each requirement was written using the EARS format to ensure they are atomic and verifiable. To check these requirements were technically feasible, they were decomposed into system requirements in Section 4. Each system requirement is mapped to a specific verification method, providing a clear framework to prove the final implementation meets all client needs.

2.2 Assumptions and Constraints

The following assumptions and constraints simplify the project. While these are implemented to ensure the model remains computationally feasible within the project timescales, the architecture remains flexible enough to allow for the removal of these simplifications should development progress exceed the initial schedule.

- **Zones:** The airport is assumed to have distinct zones. Aircraft move instantly between them.
- **Constant Speed:** Every aircraft is assumed to travel at a pre-determined constant speed. Aircraft descending in the holding pattern do so instantly.
- **Constant Fuel Burn:** All aircraft consume fuel at a constant rate, regardless of speed, altitude or weight of the aircraft.
- **Perfect Pilots:** Assume pilots follow the model perfectly every time, there is no human error, so no failed landings leading to go-arounds for example. [3]
- **Zero Security Threats:** The system is not considered to present a security risk as it will not handle any sensitive data.
- **Infinite Holding Queue Capacity:** While real-world holding stacks are finite, this model assumes the available airspace allows for infinite vertical expansion to simplify the logic and prioritise calculating hold times.
- **Scope Exclusions:** The project is limited to aircraft movements and does not require modelling of terminal activities such as baggage handling, gate assignment, or passenger movement. Helicopter movements are also outside of scope.

3 System Operational Modes

This section defines the behavioural logic of the application and illustrates how the system changes state in response to user input. To support the client's request for comparing results across multiple airport configurations, the architecture distinguishes between the Simulation Lifecycle and the Results Archive Comparison Mode. This separation allows users to access the comparison interface both immediately following a simulation run or by bypassing the simulation engine entirely to navigate directly from Home Mode, supporting UR-FR-34. By facilitating the comparison of results without the time and computational overheads of re-initiating a simulation, the design significantly improves user workflow efficiency. Furthermore, structuring the system into distinct operational modes improves modularity, making the system more maintainable and directly addressing SR-NFR-1.

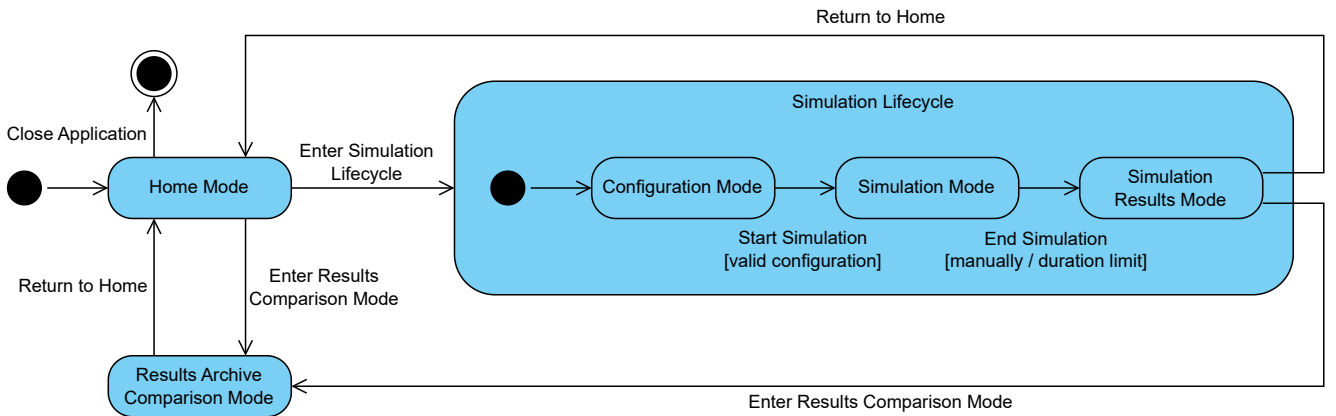


Figure 1: State machine diagram, showing the system modes and how the application moves between them.

4 System Requirements Specification

This section decomposes the user requirements into technical specifications, mapping each system requirement (SR) back to its parent user requirement(s) (UR) for full traceability and establishing specific verification methods for testing.

Table 2: System requirements. Note: IDs in this table correspond to the SR-FR-x prefix (e.g. 1 = SR-FR-1). UR refers to the linked user requirement (e.g. 1 means linked to UR-FR-1).

ID	UR	System Requirement	Verification Method
Configuration Mode			
1	1, 2, 3, 4, 5, 7	Accept integer and enumerated inputs for runway count (1-10), flow rates (0-100), runway modes and statuses and max wait time, validating range constraints before instantiation.	Unit Test: runwayCount {0,11}, flowRate{-1, 101}, maxWaitTime {-1} rejected with error; {1,10}, {0, 100}, {0, 300} accepted. Verify enumeration type checks for runwayMode and runwayStatus.
2	6, 9	Parse user-defined schedules for runway changes and emergencies into an event queue, sorted by simulation timestamp. If enabled by user, automatically generate events and add them to the event queue during the simulation.	Integration Test: upload schedule with events at T = 10, T = 5; verify EventQueue sorts as 5, 10. System Test: Enable auto-generation; verify new events appear in EventQueue during runtime.
3	8	Serialise valid configuration objects to the local disk using JSON format to enable storage and retrieval of simulation setups.	Inspection: Save configuration; verify config.json is created. Unit Test: Modify JSON with invalid syntax; verify system handles JsonParseException.
Simulation Mode			
4	10, 12	Instantiate simulation engine with user-defined configuration using the Java Spring Boot framework, incrementing system state in discrete time steps.	System Test: Initialise Spring application and verify all parts load using configuration data. Verify tick() increments global clock precisely.
5	11, 17	Instantiate arriving and departing aircraft objects at uniform intervals derived from flow rates, applying a Normal randomisation ($\sigma = 5\text{min}$) to vary scheduled times. Assign initial fuel to arrivals uniformly distributed between 20-60min. For departures, fuel is not modelled, so set it to Null.	Statistical Analysis: Generate 1,000 aircraft; verify scheduled times follow the uniform distribution with Normal randomisation. Unit Test: verify Departure.fuel == Null and Arrival.fuel $\in [20,60]$.
6	14, 18	Implement priority queues for landings (ordered by emergency status (and fuel level to determine order between emergency status aircraft), then arrival time) and FIFO queues for departures.	Unit Test: Queue three arrivals: {A: Emergency, 15min fuel}, {B: Emergency, 12min fuel}, {C: None, 20min fuel}. Verify queue order is B, A, C.
7	13, 19, 20, 22	Implement a resource locking mechanism to ensure only one Aircraft object occupies a Runway object per tick, enforcing minimum occupancy durations.	Integration Test: Attempt to move Aircraft2 to Runway1 while Runway1.isOccupied. Verify Aircraft2 remains in queue, runway state unchanged.
8	15, 16	Enforce logic to divert aircraft before fuel $\leq 10\text{min}$ and ensure aircraft in holding pattern maintain a minimum vertical separation of 1000ft.	Unit Test: Inject aircraft with fuel = 10.1 and 10.0; verify divert() triggers at ≤ 10 . Unit Test: Attempt to add aircraft at 1500ft to holding queue containing aircraft at 1000ft; verify it is rejected.
9	21	Implement a scheduling algorithm for Mixed Mode runways that alternates slot allocation between arrival and departure queues, prioritising emergency arrivals.	System Test: In mixed mode, queue Arrival1, Departure1, Arrival2. Inject an emergency arrival, verify sequence shifts to Emergency, Departure1, Arrival1, Arrival2.
10	23, 27, 28	Provide runtime controls to pause, resume, modify tick duration and inject state changes (e.g. aircraft/runway status changes) into the simulation.	System Test: Trigger pause(); verify simulation pauses. Inject state changes; verify simulation queues re-sequence.
11	24, 25, 26, 29	Render a real-time view of simulation data, including a graphical representation of aircraft positions and status tables, using HTML Canvas and JavaScript.	System Test: Inspect interface and verify JavaScript correctly draws aircraft coordinates from Spring Boot data. Also, verify table rows are correct and update dynamically.
Output Modes			
12	30, 31, 32, 33, 35	Aggregate performance data into a statistics object, providing additional information such as variance if additional metrics are enabled.	System Test: Create configuration and run simulation; manually calculate statistics and verify system output matches.
13	34	Serialise simulation results to local storage in JSON format to allow comparison of results in the Results Archive.	Inspection: End simulation and verify results.json is created with correct data. Verify Results Archive can parse and display the file.

5 Non-Functional Requirements

This section outlines the non-functional requirements (NFRs), defining the quality attributes and performance standards of the system. Both User (UR) and System (SR) non-functional requirements are presented in the table below.

Table 3: User and system non-functional requirements. "NFR" is omitted from IDs: UR-x = UR-NFR-x; SR-x = SR-NFR-x.

ID	Pr.	User Requirement	Justification and Source
UR-1	M	Reliability and Accuracy: the system shall produce reproducible results for identical input parameters to ensure accurate results.	The client requires a "reliable, accurate solution", so the results can be compared to inform decision making for airport managers in the real-world.
UR-2	M	Usability and Error Handling: the UI shall be intuitive and provide user guidance, including user-friendly yet informative error messages.	UIs "need to be intuitive" and must "consider how the end-user is guided to enter the input variables" to prevent invalid states.
SR-1	M	Maintainability: the system shall be developed using modular design patterns, including the MVC architectural pattern.	Spec: "Systems must consider future maintainability", MVC architecture ensures components can be updated independently.
UR-3	S	Performance: the simulation should be efficient so it can run at accelerated speeds.	Ensures "averages are accurate" within a reasonable time for the user.
SR-2	C	Scalability: the system architecture shall support future expansion, ensuring increasing runway or flow rate limits requires only minor modifications.	Considers "flexibility" and would allow program to be edited in future to model more intensive scenarios.
SR-3	W	Security: the system shall not implement encryption or authentication methods.	"This system is not considered to represent a security risk" so these methods are not required.

6 Team Roles, Organisation and Management

Table 4: Team roles with explanations of roles and justifications of role selections.

Role	Explanation of Role Responsibilities	Reason for Role Selection
Josh - Project Manager	Risk mitigation, project planning, execution and monitoring. Ensures delivery meets requirements and deadlines while maintaining team cohesion.	Organised leader with strong decision-making skills. Good leadership and communication skills to help maintain team cohesion.
Abel - Software Architect	Defines system structure via UML class and behavioural diagrams to eliminate ambiguity concerning system operation and functionality.	Detail-oriented and strategic; will use past experience to construct architecture and translate requirements into functional systems.
Leroy - Designer	Designs interfaces for configuration and real-time simulation views, focusing on intuitive input controls and visual representations.	Experience in UI/UX and web design; applies UI/UX principles to ensure high system intuitiveness and usability.
Lin - Designer	Designs interfaces for simulation results and comparison views, focusing on clear data visualisation and intuitive interfaces for comparing results.	Web development background provides a strong foundation in UI layout and usability principles for data-heavy applications.
Rohayl - Software Developer	Implements core simulation logic, including aircraft queues and runway handling, to turn user and system requirements into functional code.	Experience in developing back-end-heavy web-based applications. Skilled at decomposing problems into manageable subtasks for incremental development.
Ben - Software Tester	Validates system behaviour against requirements, generating test cases. Identifies and resolves problems during integration.	Strong attention to detail; enjoys analysing and debugging complex systems and identifying boundary conditions and inconsistencies.

While roles define core responsibilities, they are not restrictive. Team members are encouraged to maintain awareness of the entire system and contribute to any task once primary responsibilities are met. Conversely, two designers were appointed to address the specification's emphasis on intuitive user interfaces. By assigning two people, we ensure sufficient resources to produce a high-quality user experience. This organisation guarantees that key roles and important customer requirements are satisfied, while providing the flexibility for members to contribute to software development, ensuring sufficient manpower to implement the product within the time constraints. Furthermore, this adaptable structure allows the team to respond effectively to any delays, technical challenges or changing customer requirements throughout the project lifecycle.

The project utilises an Agile-inspired methodology with two-week sprints. Management is collaborative and informal, prioritising open communication over rigid hierarchy. The Project Manager ensures timely task completion and resolves team issues through two high-quality meetings per week. Meetings are kept focused using structured agendas, and kept impactful by ending with clear action points detailing responsibilities and deadlines.

7 References

- [1] Alistair Mavin et al. “Easy Approach to Requirements Syntax (EARS)”. In: *2009 17th IEEE International Requirements Engineering Conference*. 2009, pp. 317–322. DOI: 10.1109/RE.2009.9.
- [2] International Air Transport Association (IATA). *The busiest hour at the busiest airport in the world*. Tech. rep. Chart of the Week, published 21 July 2023. IATA Sustainability and Economics, July 2023. URL: www.iata.org/economics.
- [3] SKYbrary. *Go-Around*. <https://skybrary.aero/articles/go-around>. Accessed: 2026-02-08. 2026.